LLM-Written Detection Rules Are Precise but Incomplete: A Pilot in CAGE-4 with Negative Results on Co-Evolutionary Purple Teaming

Bhavit Anand*
Maharaja Agrasen Institute of Technology (GGSIPU), Delhi, India
bhavitanand.dev@gmail.com

Krish Anand*
Maharaja Agrasen Institute of Technology (GGSIPU), Delhi, India
krishanand.dev@gmail.com

Prateek Tanwar*
Maharaja Agrasen Institute of Technology (GGSIPU), Delhi, India
tanwarprateek07@gmail.com

* All authors contributed equally to this work.

Abstract

Detection rules are the working material of a security operations centre (SOC), and they age quickly because attackers change their behaviour once a rule starts to fire. We ask whether a team of language-model agents can write and maintain Sigma-style detection rules in a closed loop with an adaptive attacker. On the CAGE Challenge 4 enterprise network we built a telemetry bridge that turns simulator observations into rule-matchable events with hidden ground-truth labels, five LLM Blue agents that propose validated rules every round, and a Red agent that adapts its plan to how much of its activity was caught. Frozen rule sets from ten training runs per condition were scored on ten held-out seeds against five Red strategies, one of them unseen. The LLM-written rules are precise (native precision of at least 0.97, false-positive rate of at most 0.03) but incomplete: native recall is 0.49 for the co-evolved system and 0.26 against a fixed Red. Co-evolution is the hypothesis we tested and did not confirm: its recall advantage over a fixed attacker did not replicate on ten further runs per condition (+0.076 over all twenty, with an interval that includes zero), and neither a scripted evasive Red nor a larger rule budget changed that. Performance feedback traded recall for precision. Pooling the rules of independent runs raises recall steadily (0.69, 0.82 and 0.96 for three, five and ten runs, at false-positive rates of 0.024, 0.045 and 0.080), and every test episode is detected within a median of 0 to 6.5 steps. A baseline check shows that the testbed is easy to separate (attackers are 84% of native events): rules fitted without an LLM reach recall 0.94 to 1.00 at no false positives. We report the trade-offs, an event-level account of what the rules miss, and the negative results as a reproducible pilot with released code, rule sets and test episodes.

Keywords: purple teaming, detection engineering, large language models, co-evolution, Sigma rules, security operations, CAGE Challenge 4

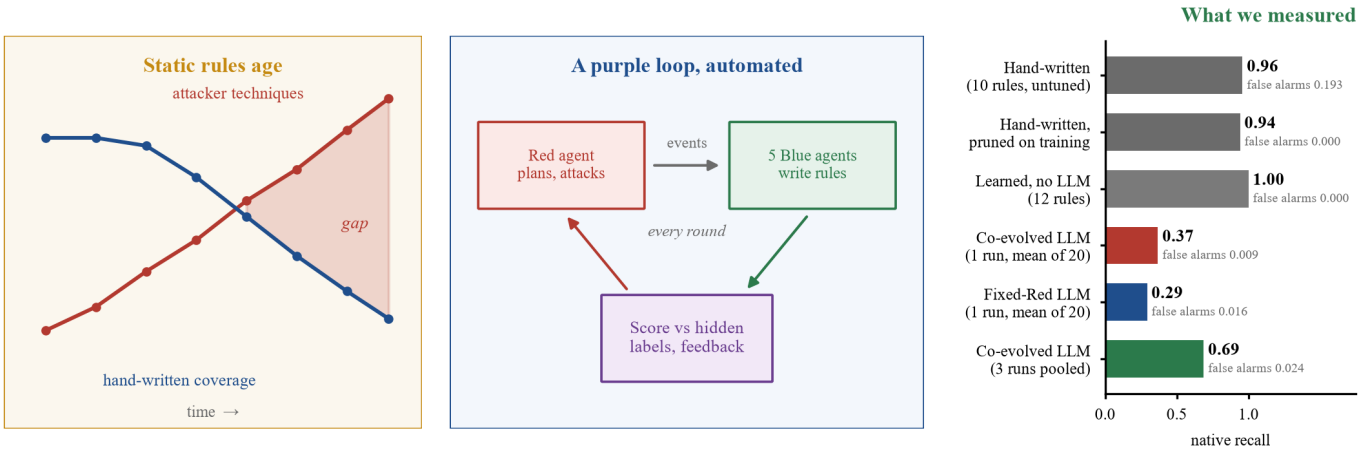


Figure 1: The idea at a glance. Left: rules written once age as attackers change (illustration). Centre: the automated purple loop studied here. Right: native recall of frozen rule sets on held-out episodes, with the false-alarm rate of each (numbers from Tables 1 and 16 and Section 6.7). The grey bars are rule sets written by hand or fitted without an LLM: on this testbed they reach higher recall than the LLM rule sets, and the fitted ones do so with no false alarms. The single-run LLM bars average all twenty runs (ten original, ten replication).

1 Introduction

Detection engineers write rules that turn raw telemetry into alerts, and attackers then change their behaviour until the rules stop firing. Keeping up with that loop is slow and manual: rules are written by experts, need constant maintenance, and differ between SIEM platforms [5], [6], while attacker techniques are catalogued in frameworks such as MITRE ATT&CK [17]. Large language models (LLMs) can already draft detection rules from threat reports [5]-[8], and they can act as defenders in simulated networks [2], but these studies evaluate the defender against attacks that do not react to it. A rule set that looks strong on a fixed trace can fail against an attacker who has seen what the rules catch.

Purple teaming addresses this in practice by putting the attacking and defending teams into one feedback loop. This paper

asks whether a small version of that loop can be automated end to end. An LLM-driven Red agent adapts its plan to what the defender detects, and LLM-driven Blue agents revise executable, human-readable detection rules from what they missed and what they flagged wrongly. We build the loop on CAGE Challenge 4 (CAGE-4) [9], built on the CybORG gym [23], a public enterprise-network simulation, and evaluate frozen rule sets on held-out seeds and on an attacker strategy never seen in training.

The study is deliberately a pilot. It uses a 4-billion-parameter local model, ten training runs of eight rounds per condition, and a detection-only defender whose rules raise alerts but do not yet change the simulated network. Part of the telemetry is synthetic, because CAGE-4 does not log what Sigma-style rules need to match on. We therefore treat the results as measurements of a method on a testbed, not as evidence about production SOCs, and we report native and synthetic events separately throughout.

We organise the evaluation around six research questions.

- **RQ1 (validity and precision).** Can LLM agents write valid, precise Sigma-style rules from telemetry alone?
- **RQ2 (coverage).** How much of the attack do their rules detect, compared with hand-written and random rule sets?
- **RQ3 (feedback and co-evolution).** Does performance feedback, or training against an adaptive Red instead of a fixed one, improve the rules?
- **RQ4 (adaptation).** Does the LLM Red planner respond to the defender, and does a stronger, evasion-rewarded Red change the answer to RQ3?
- **RQ5 (robustness).** Do the conclusions hold for a larger model and for a larger rule budget?
- **RQ6 (baselines).** Do rule sets fitted without an LLM match the LLM-written rules, and how easy is the testbed?

Contributions.

- A round-based co-evolution loop in which an LLM planner drives an adaptive Red agent and five LLM Blue agents write validated Sigma-style rules once per round, with cheap executors handling individual simulation steps (Section 4).
- A telemetry bridge that converts CAGE-4's native observations into rule-matchable events, with every event tagged native, enriched, or synthetic so that the share of invented telemetry is reported, and with ground-truth labels attached at the moment the simulator creates the underlying record (Section 4.2).
- A pilot evaluation of frozen rule sets on held-out seeds and an unseen Red strategy, with six conditions, a no-feedback ablation, bootstrap differences between conditions, and a breakdown of missed attacker events by action and event type (Section 6).
- An exploratory finding, replicated on twenty runs per condition, that pooling the rules of independent runs raises recall at a false-positive rate well below that of hand-written rules (Section 6.6), and a set of negative and mixed findings that bound what such a loop can claim: feedback buys precision at the cost of recall, the LLM Red planner's plans follow the round number, and a scripted evasive Red does not show a benefit of co-evolution (Sections 6 and 8).
- A baseline check: rule sets fitted without an LLM reach recall 0.94 to 1.00 at no false positives because attackers are 84% of native test events (Section 6-G). Campaign-level detection is in the main results: every rule set flags every test episode within a few steps (Section 6.2).

2 Related Work

Rule generation with LLMs. RulePilot turns an analyst's description into SIEM queries through an agent with an intermediate representation and a reflection step [5]. AutoSigma and LLMCloudHunter extract Sigma-style rules from threat reports [6], [7], and ThreatPilot additionally generates attack commands to check the rules against [8]. CTI-REALM benchmarks agents on rule generation against real telemetry [10]. These systems start from a report and produce rules once. None of them faces an attacker that responds to the rules.

LLM and co-evolutionary agents on CAGE-4. Castro et al. use LLMs as CAGE-4 defenders that choose actions, with a scripted Red [2]. Hemberg et al. evolve Red and Blue team controllers with grammatical evolution and an LLM-supported mutation operator, and find that two-sided co-evolution lowers both the highs and the lows of each side's performance and induces fluctuations [1]. Their agents output game actions rather than rules, and they list out-of-sample evaluation as future work. Shaikh et al. build an LLM-plus-RL Red agent against CAGE-4 defenders and show that stand-alone LLM attackers struggle to sustain multi-stage campaigns [3].

Red-versus-Blue loops with LLMs. Several recent systems pair an attacker and a defender around LLMs themselves. PAD alternates an attacker and a defender to align models against jailbreaks [4]. AutoPT frames purple teaming for LLM security as an autonomous red and blue loop with coverage-guided exploration [12], BlueCodeAgent lets automated red teaming feed a blue-teaming agent for code generation [13], Co-RedTeam orchestrates multi-agent vulnerability discovery and exploitation [14], and ACEA connects red and blue LLM systems in a head-to-head arena [15]. Du describes LLM Red and Blue agents that improve through prompt augmentation in a small testbed [11]. These works target model safety, code security, or exploitation. Our setting is network defence, and our Blue artefact is an inspectable rule set that can be scored event by event.

Co-evolution. Competitive co-evolution has a long history in evolutionary computation. Hillis showed that co-evolving parasites improve an optimisation procedure [20], cooperative co-evolution splits a problem across populations [21], and well-known failure modes such as disengagement and cycling are part of that literature. We borrow the vocabulary but not the machinery: our two sides alternate once per round and learn through prompts and a retirement rule, not through populations.

Position. We combine an adaptive Red, LLM-written executable rules, hidden ground-truth scoring, and evaluation of frozen rules on held-out seeds and an unseen attacker, inside one simulator. Each ingredient exists in prior work; we are not aware of a study that evaluates them together, although our literature check was not a systematic review.

3 Problem Statement and Threat Model

3.1 Problem formulation

An episode produces a stream of events $e_1, ..., e_T$, each with Blue-visible fields x_t and a hidden binary label y_t that is 1 when the event was caused by Red. A rule set R maps an event to an alert, a(e) = 1 when any rule in R matches x_t. Quality is measured per event:

$$Precision = TP / (TP + FP)$$
$$Recall = TP / (TP + FN)$$
$$FPR = FP / (FP + TN)$$

where TP counts alerts on Red-caused events, FP alerts on benign events, FN missed Red-caused events, and TN silent benign events. A Blue team improves its rule set over rounds r = 1, ..., 8 using feedback computed from the labels on training episodes only. The final rule set is frozen and evaluated on held-out test seeds without any feedback. The question is how recall, precision and FPR of the frozen set depend on how the rules were produced.

3.2 Threat model and assumptions

Red can take the ten CAGE-4 Red actions on the simulated enterprise network (discovery, exploitation, privilege escalation, impact, degradation, withdrawal, and sleep) and chooses a plan once per round. The adaptive Red sees only the share of its activity that was caught in earlier rounds (and, in one condition, the share per action); it never sees the rules. Blue sees only the sixteen visible event fields. Ground-truth labels are used for training feedback and for scoring, and are never part of what Blue or the frozen rules see at test time. We assume labelled training episodes, as they would exist in a purple-team exercise, and we assume the simulated telemetry is a fair stand-in for the fields a SOC would match on, which we cannot check.

Out of scope are real-world deployability, analyst workload, adversaries who tamper with the rule pipeline, and any response action by Blue. Blue here is a detector, not a defender that changes the network.

4 System Design

4.1 Environment

CAGE-4 simulates an enterprise network of several security zones. Five Blue agents defend it (four hold one subnet each, the fifth holds three), Green agents generate benign work, and Red agents try to reach and disrupt servers. Red's action space has ten actions: network and service discovery (DiscoverRemoteSystems, AggressiveServiceDiscovery, StealthServiceDiscovery), DiscoverDeception, ExploitRemoteService, PrivilegeEscalate, Impact, DegradeServices, Withdraw, and Sleep. An episode is 500 steps. Blue's native observation is a set of per-host records, not log lines, so a rule language needs an extra layer on top of it. Fig. 2 gives an overview of the loop.

4.2 Telemetry bridge

The bridge converts Blue's native observations into events with sixteen Blue-visible fields (event category, process and parent process, privilege, source and destination address, destination port, protocol, service name, decoy flag, host, subnet, and zone). We measured on 1,500 steps of passive-Blue episodes which Red actions leave a native record. Only ExploitRemoteService and the two service-discovery actions do. PrivilegeEscalate, Impact, DegradeServices, DiscoverRemoteSystems, and DiscoverDeception leave none.

Every event carries a provenance tag. *Native* events come directly from CAGE-4's records (connections with real addresses and ports, session-creation records, and Green's false alerts). *Enriched* fields, such as process names and the service name for a well-known port, are chosen by a seeded hash because the simulator does not provide them. *Synthetic* events are invented for successful PrivilegeEscalate, Impact, and DegradeServices actions, so that Blue has something to detect; they are delivered to the Blue agent that defends the target host's zone. DiscoverRemoteSystems, DiscoverDeception, and Withdraw stay invisible. Ground-truth labels (whether the event was caused by Red) are attached at the moment CAGE-4 creates the underlying record, and they are never part of the Blue-visible event.

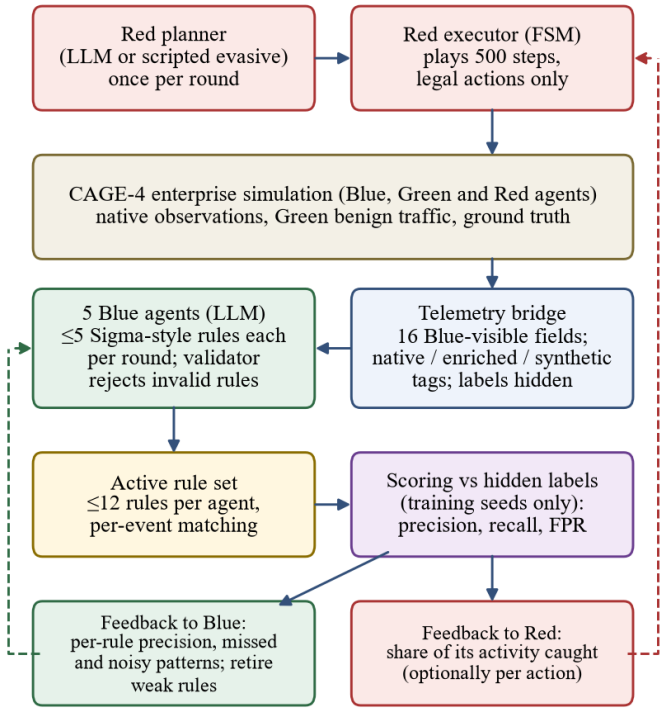


Figure 2: Overview of the co-evolution loop. Red plans and plays an episode; the telemetry bridge converts CAGE-4's native observations into rule-matchable events; five Blue agents write validated Sigma-style rules; the rules are scored against hidden ground-truth labels on training seeds only, and the scores flow back to both sides. After the last round the rules are frozen and evaluated on held-out seeds.

Fig. 3 follows one event through the system, from the simulator record to a scored alert. Fig. 4 shows a real rule written by a Blue agent, and Fig. 5 gives the training procedure of one run as pseudocode.

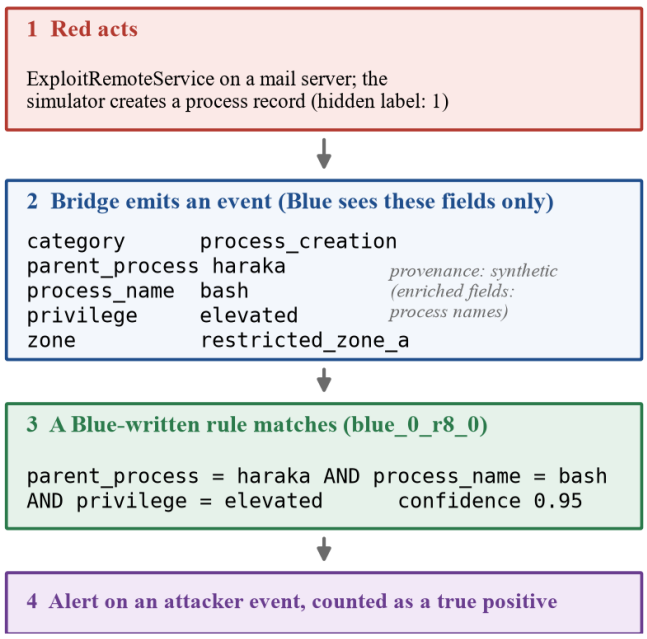


Figure 3: A worked example (illustrative event, real rule). Red's exploit produces a record, the bridge turns it into an event with visible fields only, a rule written by a Blue agent matches it, and the hidden label scores the alert as a true positive.

Sigma-style rule written by a Blue agent (unedited, seed 3)

```
title: Elevated bash spawned by haraka
level: high
logsource:
  category: process_creation
selections:
  bash_from_haraka:
    process_name:bash
    parent_process:haraka
  elevated_privilege:
    privilege: elevated
condition: bash_from_haraka and
           elevated_privilege
confidence: 0.95
rationale: Haraka is a mail server;
            bash with elevated privileges is
            highly anomalous
```

Figure 4: A rule written by a Blue agent, unedited (seed 3, round 8): selections over visible fields, a condition, a confidence value and a short rationale.

```
Algorithm 1  One training run (8 rounds)
for r = 1 .. 8:
  # Blue proposes (feedback from round r-1)
  for each of 5 Blue agents:
    rules <- LLM(prompt: schema, patterns, feedback)
    keep only rules that pass the validator (<= 5)
    active <- last 12 rules (cap)
  # Red plans, then the episode is played
  plan <- Red.plan(memory)   # LLM or evasive
  events <- play_episode(plan, 500 steps)
  # Score with hidden labels (training seeds only)
  P, R, FPR <- score(active, events)
  # Feedback and retirement
  Blue.feedback <- per-rule precision, missed
                   and noisy patterns
  Red.memory <- share of its activity caught
  retire rules with >= 3 fires and < 30% precision
freeze active rules; score on held-out seeds
```

Figure 5: Algorithm 1. One training run: Blue proposes, Red plans, the episode is played, rules are scored against hidden labels on training seeds only, feedback goes to both sides, and weak rules are retired.

4.3 Rule language and validation

Rules are Sigma-style JSON objects [16] with named selections over the visible fields, a condition built from AND, OR, and NOT, a severity level, a confidence value, and a short rationale. Three field modifiers are allowed (contains, startswith, endswith). A rule judges one event at a time; there are no counts or time windows. A validator rejects rules that use unknown fields, unsupported modifiers, or malformed conditions, and invalid rules are discarded. Constrained JSON decoding keeps most proposals well formed.

4.4 Red agent

Red is an LLM planner with a cheap executor. At the start of a round the planner returns a JSON plan: a goal (for example reconnaissance or impact), a scan style (aggressive, stealthy, or mixed), how much to probe for decoys, whether to prioritise servers, and how aggressively to pursue Impact and DegradeServices. The plan is validated, retried once on error, and replaced by CAGE-4's default scripted behaviour if it stays invalid. A finite-state executor built on CAGE-4's own Red agent compiles the plan into action probabilities and plays all 500 steps, checking each action for legality. The planner sees a memory of its last rounds, including how many of its exploits succeeded and what share of its activity the defender's rules caught, and it is told that the defender learns from what it does.

4.5 Blue agents

Each of the five Blue agents calls the LLM once per round for up to five rules. The prompt contains the rule format, the field vocabulary, three valid example rules, a short statement of what the language cannot express, the agent's current rule titles, and a summary of the previous round. That summary always lists the most common event patterns seen in the agent's zone (unlabelled). In the feedback conditions it also states how many attacker and benign events the rules matched, per-rule precision, the attacker patterns that no rule matched, and the benign patterns that fired alerts. These statements are computed from ground-truth labels, so they are used only on training episodes. An agent keeps at most twelve active rules, and after every round a rule that fired at least three times with fewer than 30% of its alerts on attacker activity is retired.

4.6 Conditions

We compare six rule sources. *Static*: ten hand-written rules applied in every zone. *Random rules*: each round every agent adds five rules built from randomly chosen observed events, with the same retirement step and no LLM. *LLM, fixed Red*: Blue with feedback against CAGE-4's default Red. *LLM, co-evolved, no feedback*: Blue sees telemetry only, against the adaptive Red. *LLM, co-evolved*: Blue with feedback against the adaptive Red. *LLM, co-evolved, per-action signal*: as the previous one, but Red's memory also lists, for each of its own actions, the share of that action's events the defender's rules caught. This condition was added after the first results showed that Red's plans hardly changed (Section 6.4); it tests whether a richer signal makes Red respond. Two further conditions use a scripted evasive Red and are described in Section 6.4.

5 Experimental Setup

5.1 Training protocol

Each condition has ten independent training runs (seeds 0 to 9), each with eight rounds. A round is one 500-step episode, preceded by one warm-up episode so that the first round already has telemetry to look at. Red and Blue alternate within a round: Blue proposes, Red plans, the episode is played, and both sides receive their feedback. Training environment seeds are derived by hashing and are at least 1000 apart from the evaluation seeds, apart from round one, which uses seeds 0 to 9, so no training episode coincides with a test episode.

5.2 Evaluation protocol

After training, each run's rules are frozen and scored without feedback on test seeds 100 to 109, which are never used in training. Each rule set is scored on 50 test episodes: five Red strategies times ten seeds. Four strategies are seen in the sense that they are the scripted baseline or variants used when developing the system (the default CAGE-4 Red, aggressive, stealthy, and deception-aware); the fifth, a hit-and-run strategy that withdraws from hosts, is held out and appears only in test. Test events are generated once with a passive Blue and cached, so every condition is scored on identical events.

5.3 Metrics and statistics

Precision, recall, and false-positive rate (FPR) are computed per event against the hidden labels, overall and separately for native and synthetic events. Because synthetic events state their signal in an explicit field, recall on them is easy by construction; we therefore never pool them with native events in a headline number. For each condition we average over test episodes within a run, then report the mean and a 95% percentile-bootstrap interval over runs [19]. Differences between conditions use a bootstrap in which each side is resampled independently. With ten runs the intervals are still wide, and the static rule set has a single run and no interval.

5.4 Models and compute

The main experiments use Qwen3-4B-Instruct-2507 [18] (4-bit quantised) served by llama.cpp [22] on one laptop GPU (NVIDIA RTX 4050, 6 GB); a confirmatory check uses GLM-4.7-Flash (3-bit quantised, partly offloaded to the CPU) on the same machine. A Blue call takes about 14 seconds and returns valid rules about 94% of the time in isolation. Training rounds in the experiments reported here added up to about 46 process-hours, 14 of them for the larger model (several runs shared the one GPU at a time, so this exceeds wall-clock time), plus test-episode generation and scoring. A training round took about two minutes with the 4-billion-parameter model when four runs shared the GPU, and roughly 3 to 9 minutes with the larger model. Appendix A lists all settings.

6 Results

6.1 RQ1: validity and precision

About 91% of proposed rules were valid across the LLM conditions, and the active rule set averaged about 53 rules over the eight rounds (60 at the end of training, the cap of 12 per agent). All three LLM conditions produced precise rule sets. On native events precision was at least 0.97 and mean FPR at most 0.03 on both seen and unseen Red (Tables 1 and 2). The random-rule control, by contrast, fired on about half of benign events (FPR 0.49 on seen Red), and the static rules on about one in five (0.19). Fig. 6 summarises recall and FPR for all conditions.

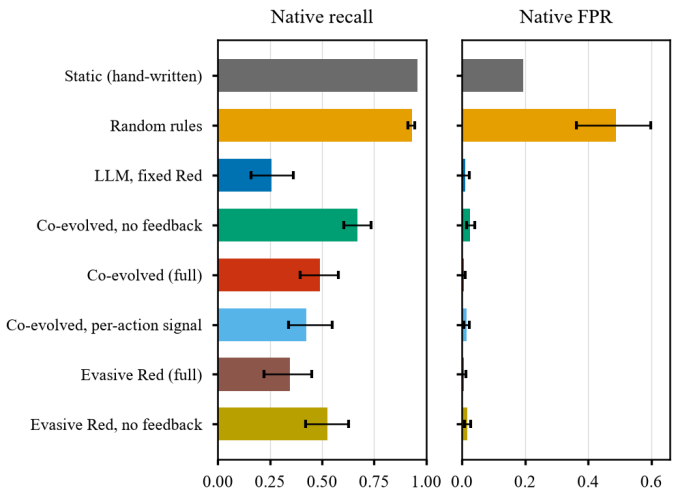


Figure 6: Native recall (left) and native false-positive rate (right) of frozen rule sets on the four seen Red strategies. Bars are means over runs; whiskers are 95% bootstrap intervals (the static rule set is a single run).

Table 1: Frozen rules on the four seen Red strategies, native events (mean [95% bootstrap CI] over training runs).

Condition	Runs	Precision	Recall	FPR
Static hand-written rules	1	0.962	0.958	0.193
Random exemplar rules + retirement	10	0.909 [0.891, 0.931]	0.932 [0.913, 0.945]	0.488 [0.364, 0.598]
LLM Blue vs fixed Red	10	0.974 [0.926, 1.000]	0.258 [0.158, 0.361]	0.011 [0.000, 0.023]
LLM co-evolved, no feedback	10	0.992 [0.988, 0.996]	0.671 [0.604, 0.736]	0.027 [0.014, 0.040]
LLM co-evolved	10	0.996 [0.993, 1.000]	0.491 [0.396, 0.578]	0.006 [0.000, 0.011]
LLM co-evolved, per-action signal	10	0.992 [0.987, 0.997]	0.426 [0.340, 0.549]	0.014 [0.007, 0.024]

Table 2: Frozen rules on the held-out hit-and-run strategy, native events.

Condition	Runs	Precision	Recall	FPR
Static hand-written rules	1	0.966	0.954	0.169
Random exemplar rules + retirement	10	0.907 [0.888, 0.929]	0.932 [0.913, 0.945]	0.477 [0.356, 0.585]
LLM Blue vs fixed Red	10	0.971 [0.918, 1.000]	0.260 [0.163, 0.348]	0.012 [0.000, 0.026]
LLM co-evolved, no feedback	10	0.991 [0.986, 0.995]	0.684 [0.602, 0.756]	0.029 [0.016, 0.043]
LLM co-evolved	10	0.994 [0.988, 1.000]	0.477 [0.375, 0.571]	0.007 [0.000, 0.014]
LLM co-evolved, per-action signal	10	0.992 [0.987, 0.997]	0.459 [0.372, 0.573]	0.015 [0.007, 0.023]

6.2 RQ2: coverage

Recall is where the LLM rule sets fall short. On native events against seen Red strategies, the co-evolved system reached 0.49 [0.40, 0.58], the fixed-Red baseline 0.26 [0.16, 0.36], and the no-feedback ablation 0.67 [0.60, 0.74]. The hand-written rules reached 0.96 and the random rules 0.93, but at the false-positive rates above. Fig. 7 shows the resulting trade-off: the LLM rules occupy the low-FPR region with partial recall, and neither the hand-written nor the random rules reach that region.

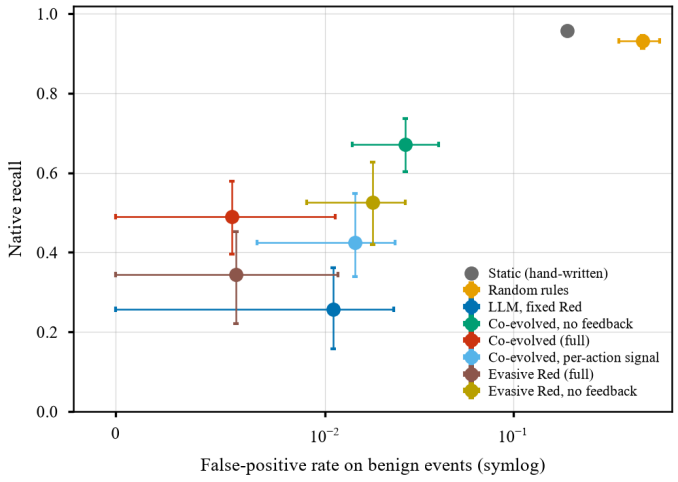


Figure 7: Recall against false-positive rate for every condition (native events, seen Red strategies; horizontal axis is symmetric-log). The LLM-written rule sets sit in the low-FPR, partial-recall region; hand-written and random rules sit at high recall with high FPR.

By attacker strategy. Native recall of every rule set was almost the same against all five Red strategies (Table 3). For the co-evolved system it ranged from 0.48 to 0.50 and for the fixed-Red baseline from 0.25 to 0.26, with the unseen strategy inside that range. We saw no extra drop on the unseen strategy, but with this sample and these intervals we cannot claim that the rules generalise either. What the LLM rule sets miss therefore depends on which event types the rules cover, not on how the attacker behaves.

Table 3: Native recall by Red strategy (mean over runs; hit-and-run is the unseen strategy).

Condition	cage4 default	aggressive	stealthy	deception aware	hit and run
Static	0.96	0.93	0.98	0.97	0.95
Random rules	0.93	0.92	0.94	0.93	0.93
LLM, fixed Red	0.26	0.26	0.25	0.26	0.26
LLM, no feedback	0.66	0.66	0.68	0.69	0.68
LLM, co-evolved	0.48	0.48	0.50	0.50	0.48
LLM, per-action signal	0.42	0.42	0.43	0.44	0.46

What is missed. Table 4 breaks native attacker events down by the action that produced them. The hand-written rules catch every exploit connection and every exploit-spawned process; the LLM rule sets catch between 27% and 75% of the exploit connections, and almost none of the processes the exploits start (9% to 19%). Scans are caught least often by the LLM rules (29% to 45%). Scan connections may resemble the benign connections of Green agents, which would make precise rules hard to write, but we did not test this.

Table 4: Share of native attacker events caught, by producing action and event type, seen Red strategies (mean over runs). Fixed: fixed Red; No fb.: no feedback; Co-evo.: co-evolved; Aware: per-action signal.

Action / event	Static	Rand.	Fixed	No fb.	Co-evo.	Aware
Aggressive scan / connection	0.67	0.81	0.29	0.45	0.31	0.32
Stealth scan / connection	0.68	0.81	0.30	0.45	0.31	0.32
Exploit / connection	1.00	1.00	0.27	0.75	0.55	0.47
Exploit / process	1.00	0.39	0.09	0.15	0.19	0.14

Synthetic events. Recall on synthetic events was 0.53 for the co-evolved system, 0.73 for the ablation, and 0.61 for the fixed-Red baseline, while the static rules reached 1.00; precision was 1.00 for all conditions. That a four-billion-parameter writer misses nearly half of events that announce themselves (an elevated privilege or a stopped service) suggests that the limit is rule coverage per agent and zone rather than the difficulty of the signal, although we did not test this directly.

Campaign-level detection. Event recall understates how reliably an attack is noticed. In all 40 seen test episodes every rule set raised at least one alert on an attacker event. The median delay between the first attacker event and the first alert, averaged over runs, was 0.0 to 0.2 steps for the hand-written, random and no-feedback sets, 2.0 steps for the co-evolved set, 3.5 for the per-action set and 6.5 for the fixed-Red set (Appendix C, Table 14). So rule sets with 26% to 67% event recall still flag every campaign within a few steps in this simulator; we did not examine why, and a real network may differ.

6.3 RQ3: feedback and co-evolution

The interval for the co-evolved system does not overlap that of the fixed-Red baseline, so co-evolution with a varied Red improved recall in these ten runs (the replication below qualifies this). The no-feedback ablation was higher than the full system and its interval does not overlap either, so on recall the feedback loop hurt rather than helped, and at this rule cap it bought a lower false-positive rate instead (Section 6.5 examines a larger cap). Table 5 gives the differences with bootstrap intervals: the co-evolved system is 0.23 above the fixed-Red baseline and 0.18 below the no-feedback ablation, and both intervals exclude zero. Giving Red a per-action detection signal changed little (0.43 [0.34, 0.55]; 0.07 below the standard co-evolved system, with an interval that includes zero).

Table 5: Differences between conditions on seen Red strategies, native events (mean difference with 95% bootstrap CI over runs).

Comparison	Recall	FPR	Precision
Co-evolved minus fixed Red	+0.233 [+0.093, +0.365]	-0.005 [-0.019, +0.006]	+0.023 [-0.004, +0.071]
Co-evolved minus no feedback	-0.181 [-0.293, -0.070]	-0.021 [-0.036, -0.007]	+0.004 [-0.001, +0.009]
Per-action signal minus co-evolved	-0.065 [-0.194, +0.082]	+0.009 [-0.001, +0.018]	-0.004 [-0.010, +0.002]
Per-action signal minus no feedback	-0.246 [-0.362, -0.113]	-0.012 [-0.028, +0.003]	+0.000 [-0.006, +0.006]

Replication. Because the co-evolution advantage was a central claim, we trained ten further runs per condition (co-evolved and fixed Red, seeds 15 to 24, same settings) and scored them on the same test episodes. The ordering reversed: native recall was 0.246 for the co-evolved runs and 0.327 for the fixed-Red runs (difference -0.081 [-0.212, +0.050]), with FPR 0.013 against 0.021. Over all twenty runs per condition the co-evolved system had recall 0.368 and the fixed-Red baseline 0.293, a difference of +0.076 [-0.035, +0.184], and FPR 0.009 against 0.016 (-0.007 [-0.016, +0.001]). The recall advantage in Table 1 is therefore best read as an optimistic first estimate, and the lower false-positive rate is the more consistent difference, although its interval also touches zero. The no-feedback ablation and the other conditions were not replicated.

Training dynamics. Fig. 8 shows native recall, FPR, and precision on the training episode of each round. Against the fixed Red, native recall fell from 0.61 in round 1 to 0.25 in round 8 as the feedback pushed Blue towards narrow, precise rules and retired the rest. With the adaptive Red, recall fell with feedback (0.65 to 0.50) and stayed flat without it (0.69 to 0.68). Random rules rose from 0.86 to 0.93 while accumulating about 54 active rules on average over the rounds.

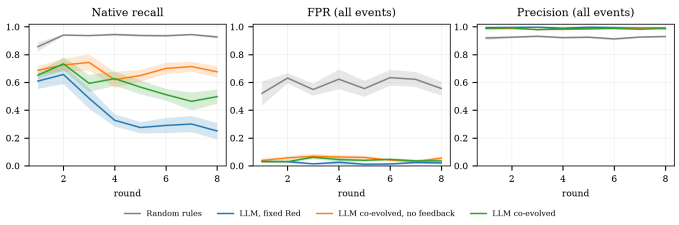


Figure 8: Native recall, FPR (all events), and precision (all events) on the training episode of each round: mean over ten runs, shaded band is plus or minus one standard error.

6.4 RQ4: does Red adapt?

LLM planner. The Red planner fell back to its default plan in one of its 160 planning calls. Over the 80 rounds of each adaptive condition it chose a stealthy scan mix in 41 or 42 rounds and a mixed one in 38, and never an aggressive one; its stated goals were balanced (38 or 39 rounds), decoy evasion (31), and stealth reconnaissance (10). The goal depended mainly on the round number: in round 1 every run chose decoy evasion, and in every later round the ten runs of a condition used at most two distinct goals. The two conditions produced nearly the same plans, so in this pilot Red's choices did not visibly depend on what Blue was doing. The per-action signal did not change this: with it, Red again chose decoy evasion in round 1 in all ten runs, used at most two goals in each later round, and fell back to the default plan once in 80 calls. We therefore describe the adaptive Red as a varied attacker whose plans follow the round number, rather than as evidence of an arms race.

Scripted evasive Red. Because the language-model planner did not respond to Blue, we added a scripted Red that adapts by construction. It holds four fixed plans (quiet probing, quiet push, loud push, mixed), tries each once in a random order during rounds 1 to 4, and then replays the plan with the highest reward, where reward is the share of its activity left uncaught and is zero if it achieved no successful impact. The design, the success criterion and the test were fixed before any result was seen: tune on no seeds, run once on seeds 0 to 9, score once on the test seeds, and call the claim supported only if the co-evolved recall minus the fixed-Red recall has a 95% bootstrap interval above zero without a material rise in FPR. We trained Blue with feedback against this Red and, as an ablation, Blue without feedback (ten runs each, same 4-billion-parameter model). Table 6 gives the result on the seen strategies. The criterion was not met. Native recall for Blue trained against the evasive Red was 0.34 [0.22, 0.45], against 0.26 for the fixed-Red baseline; the difference was +0.087 with an interval of [-0.072, +0.241], which includes zero. FPR did not rise (0.006). The blind ablation again reached the highest recall, 0.53, and the evasive-Red Blue was 0.182 below it (interval [-0.336, -0.027]). Whether Red responded is mixed. The share of Red activity caught during training was 0.70 and 0.78 in rounds 1 and 2 and between 0.33 and 0.46 in rounds 4 to 8 with feedback, whereas without feedback it stayed between 0.60 and 0.76; but the plan chosen in the exploit rounds changed in only 8 of 30 consecutive pairs with feedback (9 of 30 without), and no plan was clearly better for Red (mean share caught 0.44 to 0.57 with feedback). So the drop in caught share mostly reflects Blue learning, and we have no evidence that a Red that learns to evade makes Blue better. We therefore do not claim that co-evolution improves detection over training against a fixed attacker.

Table 6: Evasive-Red test, seen strategies, native events, ten runs per condition (mean with 95% bootstrap CI over runs).

Condition	Recall	FPR
Fixed Red	0.258 [0.158, 0.361]	0.011 [0.000, 0.023]
Co-evolved (LLM Red)	0.491 [0.396, 0.578]	0.006 [0.000, 0.011]
Co-evolved vs evasive Red	0.344 [0.221, 0.452]	0.006 [0.000, 0.012]
Evasive Red, no feedback	0.526 [0.420, 0.626]	0.018 [0.009, 0.027]

6.5 RQ5: robustness

To check whether the 4-billion-parameter writer drove the recall problem, we repeated the three LLM conditions with GLM-4.7-Flash (a larger mixture-of-experts model, 3-bit quantised, same prompts and settings) for three training runs each and scored them on the same frozen test episodes. Three runs are too few for firm conclusions, and the intervals in Table 7 are wide; we report it as a confirmatory check only. Recall was clearly higher than for the small model against a fixed Red (0.66 against 0.26) and similar without feedback (0.69 against 0.67), but it was not higher for the full co-evolved system (0.59 against 0.49 with overlapping intervals). Native precision stayed at or above 0.98 and FPR at or below 0.07. The ordering seen with the small model did not reproduce: here the co-evolved system had the lowest native recall of the three and its interval overlaps the fixed-Red baseline, so we cannot claim a benefit of co-evolution over a fixed attacker with this model. The false-positive rate was again lowest with feedback (0.019 against 0.072 without).

Table 7: Larger writer model (GLM-4.7-Flash), three runs per condition, native events, frozen rules (mean with 95% bootstrap CI over runs).

Condition	Seen recall	Seen FPR	Unseen recall
Fixed Red	0.663 [0.452, 0.772]	0.034 [0.000, 0.090]	0.652 [0.433, 0.794]
No feedback	0.692 [0.518, 0.886]	0.072 [0.000, 0.129]	0.696 [0.579, 0.883]
Co-evolved	0.588 [0.490, 0.747]	0.019 [0.000, 0.030]	0.581 [0.501, 0.709]

Larger rule budget. The final variant, chosen on development seeds (Appendix B), raises the cap on active rules per agent from 12 to 20, which gives about 99 active rules per run instead of 60. We ran it for the fixed-Red, no-feedback and co-evolved conditions with ten runs each (seeds 0 to 9) and scored it once on the test seeds. Table 8 gives the result. Native recall rose from 0.26 to 0.40 for the fixed-Red baseline and from 0.49 to 0.54 for the co-evolved system, with FPR unchanged (0.010 and 0.007). Neither increase is clearly separated from zero (+0.140 [-0.003, +0.277] and +0.052 [-0.098, +0.202]). At cap 20 the co-evolved system was 0.145 above the fixed-Red baseline, with an interval of [-0.004, +0.303] that only just includes zero. The benefit of co-evolution over a fixed attacker therefore shrinks when the rule budget is larger, and we cannot call it established. Without feedback, recall rose from 0.67 to 0.72 (+0.051 [-0.067, +0.162]) and FPR fell from 0.027 to 0.012. At cap 20 the co-evolved system with feedback was still 0.179 below the no-feedback ablation in recall (interval [-0.327, -0.028]), but the difference in FPR (-0.005 [-0.017, +0.005]) was no longer clearly below zero, so the lower false-positive rate that feedback bought at cap 12 is not clearly present at cap 20. The development screen had suggested a larger gain for the co-evolved system (+0.084) than the final test showed (+0.052), which is a reminder that five-seed screens are noisy. A larger rule set is not free in practice, since each rule must be reviewed and maintained.

Table 8: Effect of a higher cap on active rules per agent, native events, seen Red strategies (mean with 95% bootstrap CI over ten runs).

Condition	Recall	FPR	Precision
Fixed Red, cap 12	0.258 [0.158, 0.361]	0.011 [0.000, 0.023]	0.974 [0.926, 1.000]
Fixed Red, cap 20	0.397 [0.292, 0.492]	0.010 [0.003, 0.017]	0.994 [0.989, 0.998]
No feedback, cap 12	0.671 [0.604, 0.736]	0.027 [0.014, 0.040]	0.992 [0.988, 0.996]
No feedback, cap 20	0.722 [0.624, 0.813]	0.012 [0.004, 0.023]	0.997 [0.995, 0.999]
Co-evolved, cap 12	0.491 [0.396, 0.578]	0.006 [0.000, 0.011]	0.996 [0.993, 1.000]
Co-evolved, cap 20	0.543 [0.427, 0.658]	0.007 [0.002, 0.012]	0.996 [0.992, 0.999]

6.6 Exploratory: pooling independent runs

This analysis was added after the main results were in, so it is exploratory. Individual runs miss different events, so we took the union of the rules of several independent runs of the same condition (disjoint groups of 2, 3, 5 and 10 runs) and scored it on the same frozen test episodes. A first pass on the ten main runs per condition (Fig. 9, Table 15) suggested a steep gain, but it rested on one to five groups per size, so we replicated it with ten further runs per condition on unused seeds, which gives twenty runs per condition (Table 16). Recall rose steadily with the number of runs pooled. For the co-evolved condition it went from 0.37 for one run to 0.59 for two, 0.69 for three, 0.82 for five and 0.96 for ten, while FPR grew from 0.009 to 0.018, 0.024, 0.045 and 0.080; the intervals over six, four and two groups are wide (Table 16). Ten pooled runs reach the recall of the hand-written rules (0.96) at less than half of their false-positive rate (0.080 against 0.193). The gain is not only a matter of rule count: two pooled co-evolved runs (118 rules) reached 0.59, above a single run with a cap of 20 (99 rules, 0.54), although the two intervals overlap. The first pass had also suggested that co-evolved committees beat fixed-Red committees, and the replication does not support that. With twenty runs per condition the difference in recall between co-evolved and fixed-Red committees was +0.080 [-0.105, +0.255] for groups of three and +0.057 [-0.130, +0.221] for groups of five, and the difference in FPR was -0.024 [-0.055, +0.006] and -0.027 [-0.053, +0.003], so every interval includes zero. We therefore treat the pooling gain itself as supported and any advantage of co-evolved committees over fixed-Red committees as not shown. The evasive-Red committees of the first pass behaved similarly (Table 15) but were not replicated. Pooling has a cost: about 177 rules for three runs and about 590 for ten, which must be reviewed and maintained, and about half of the rules in any single run never fire (Table 11). Adding a single LLM run to the hand-written rules changed little (recall 0.958 to between 0.968 and 0.974, FPR 0.193 to between 0.199 and 0.220), so the hand-written set's own false positives dominate.

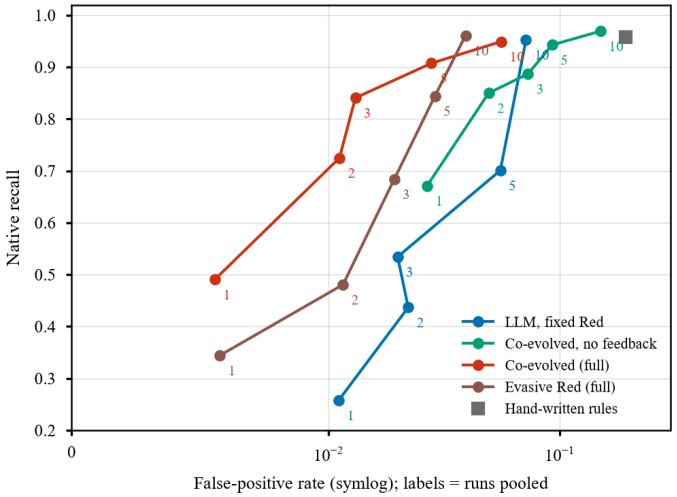


Figure 9: Recall against false-positive rate when the rules of 1, 2, 3, 5 or 10 independent runs are pooled (disjoint groups; labels give the number of runs pooled). Pooled co-evolved rule sets reach high recall at low false-positive rates. Exploratory analysis.

6.7 Baseline check: how easy is the testbed?

Baseline check. The hand-written rule set was written once and not tuned for precision, so we also asked what a fitted rule set without an LLM achieves on the same episodes. We fitted two on the training episodes only (seeds 0 to 9) and scored them once on the test seeds. Pruning the ten hand-written rules to the three that were precise on training episodes gave native recall 0.941 at a false-positive rate of 0.000 (0.937 and 0.000 on the unseen strategy). A greedy learner that picks up to twelve single-field rules gave recall 1.000 at 0.000, and 0.988 from a single training episode. Attacker events are 83.8% of native test events (about 5.2 attacker events per benign one), so raw precision is flattered: alerting on every event already gives precision 0.837. If attackers were 1 percent of events instead, precision at the observed recall and false-positive rate would be 0.744 for the co-evolved system, 0.726 for the fixed-Red system, 0.313 without feedback and 0.048 for the original hand-written set, against 1.000 for both fitted sets. The fitted sets work because a few fields separate attackers from benign events: every benign network event uses an ephemeral port, while 97.4 percent of attacker network events go to well-known ports. With the enriched fields blanked, the learned set still reaches recall 0.929 at 0.000, whereas the co-evolved rules fall from 0.49 to 0.19. We therefore read the LLM results as what an LLM-written rule set achieves on this testbed, not as evidence that it beats simpler methods; on this testbed the fitted baselines are stronger, and the comparison between pooled LLM rules (about 590 rules) and the ten hand-written rules above should be read with that in mind. A harder evaluation, with benign activity that resembles attacker activity, is the next step. The numbers are in results/pilot_baselines.

7 Case Study: What the Rules Look Like

Table 9 shows rules written by the co-evolved system in one training run (seed 3), taken unedited from the saved rule files. Three patterns stand out. First, the rules are readable and mostly sensible: they pair a process or port with a zone or privilege and give a short reason, in the style of a human rule. Second, they lean on the attributes the prompt examples emphasise (elevated privilege, parent process, uncommon ports), which explains the good coverage of privilege-related events and the poor coverage of exploit-spawned processes that carry ordinary attributes. Third, some agents write several variants of the same idea (an elevated shell under haraka or sshd appears with different parents and names), although a simple overlap measure finds few exact repeats: Table 10 shows that 6% of the co-evolved rules have a near-duplicate in the same agent's set (at least 80% overlap of matched field and value pairs), against 17% without feedback and 56% for random rules. Some rationales also assert more than the data shows, for example treating SSH on port 3390 as evasion; we did not check how often such connections occur in benign traffic. Per-rule behaviour on held-out episodes is reported in Table 11.

Table 9: Examples of rules written by the co-evolved system (seed 3). Selections are shown as field = value; the model's own confidence and reason are quoted.

Rule	Matches	Confidence and reason
blue_0_r7_2	parent_process = tomcat, privilege = elevated, process_name = sh	0.90: privilege escalation via sh is rare and suspicious in restricted zones
blue_0_r8_0	parent_process = haraka, process_name = bash, privilege = elevated	0.95: bash with elevated privileges under a mail server is highly anomalous
blue_0_r7_4	dst_port = 445, zone = restricted_zone_a	0.60: SMB access from restricted zones is uncommon
blue_2_r6_0	privilege = elevated, parent_process contains sshd, process_name contains sh/bash	0.95: SSH daemon is trusted; privilege escalation with a shell is rare
blue_4_r6_1	privilege = elevated, process_name contains python3, parent_process contains systemd	0.90: elevated python3 from systemd is rare in benign workloads
blue_0_r7_0	dst_port = 3390, service_name = ssh	0.60: SSH on non-standard ports may indicate evasion

Table 10 summarises the structure of the final rule sets over all ten runs per condition. The LLM rule sets split about evenly between process-creation and network-connection rules (43% to 52% process rules), whereas random rules are mostly connection rules (62%) with only 8% process rules, and the LLM rules contain a fixed share of service-event rules (9% to 19%). The LLM rules also carry more constraints each (2.5 to 2.9 matched field-value pairs) than random rules (1.5), which is consistent with their lower false-positive rate. Raising the cap to 20 increased the number of rules (99 per run) without changing the mix.

Table 10: Structure of the final rule sets (mean over ten runs per condition; Dup. = rules with a near-duplicate, at least 80% overlap of matched field-value pairs with another rule of the same agent; Cons. = constraints; Proc., Conn., Serv. = share of process, connection and service rules).

Rule set	Rules/run	Dup. (%)	Cons./rule	Proc. (%)	Conn. (%)	Serv. (%)
LLM, fixed Red	59.5	9	2.5	43	39	19
LLM, no feedback	58.9	17	2.6	43	44	13
LLM, co-evolved	58.9	6	2.7	51	38	11
LLM, per-action signal	59.7	10	2.7	50	41	9
Random rules	58.9	56	1.5	8	62	29
LLM, co-evolved, cap 20	99.4	15	2.9	52	39	9

We also ran every saved rule over the cached test episodes of the four seen Red strategies (40 episodes) and counted its alerts on

native attacker events, native benign events and synthetic events. Table 11 and Fig. 10 show that the nominal size of the rule sets is misleading. About half of the LLM-written rules (47% to 48% with feedback, 37% without) never fire on any test event, and only 20% to 33% of them fire on a native attacker event. A handful of rules carries the detection: three rules produce 80% of the native attacker alerts of the fixed-Red and co-evolved sets, and six do so without feedback, whereas random rules, which are mostly broad connection rules, need about 25. Almost no LLM rule alerts mostly on benign events (0% to 3%), which matches their low false-positive rate. The effective rule set is therefore much smaller than its count, and on these episodes pruning the silent rules would not change any reported metric. We did not examine why so many rules stay silent; one plausible reason is that rules combining two to three field constraints are too specific to match the events that occur, but we did not test this.

Table 11: Behaviour of individual rules on held-out episodes (seen Red strategies, mean over ten runs per condition; never fire means no alert on any native or synthetic event).

Rule set	Rules / run	Never fire (%)	Fire on native attacker events (%)	Mostly benign alerts (%)	Rules for 80% of attacker alerts
LLM, fixed Red	59.5	48	20	1	3.1
LLM, no feedback	58.9	37	33	3	6.0
LLM, co-evolved	58.9	47	26	0	3.2
Random rules	58.9	6	63	1	24.9

Fig. 10 plots how quickly the attacker alerts accumulate when rules are ranked by their number of alerts.

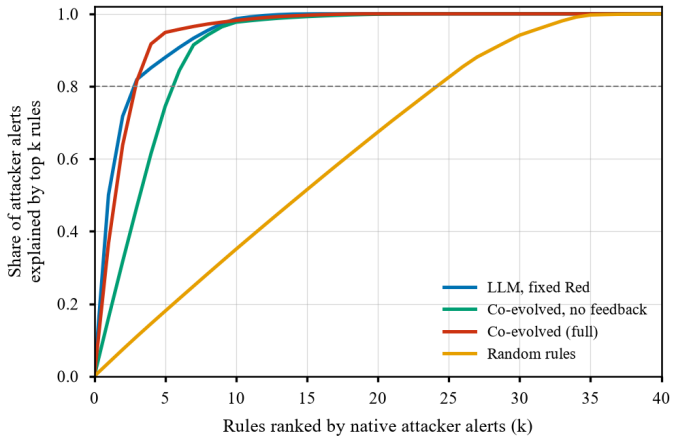


Figure 10: Share of native attacker alerts explained by the k most productive rules, averaged over ten runs per condition. A few LLM-written rules carry almost all of the detection; random rules spread it over about 25 rules.

8 Discussion

The trade-off. The clearest finding is a trade-off. Rules written by a small model are precise and quiet, and they miss most of what a broad rule set catches. Hand-written and random rules sit at the opposite corner, with high recall and high false-positive rates. Performance feedback moves the writer further towards the precise corner: with feedback the false-positive rate was lower (0.006 against 0.027) and recall lower (0.49 against 0.67) at the standard cap of 12 rules per agent. The fixed-Red training curve shows the same pull, with recall decaying as rules are retired for low precision. Recall also barely differs between Red strategies, so the rule sets are limited by what they cover, not by how the attacker behaves.

What the results support. In the ten main runs, co-evolution with a varied attacker preserved more recall than training against one fixed attacker, without costing precision. That finding did not survive replication: ten further runs per condition on new seeds gave the opposite ordering (0.25 against 0.33), and over all twenty runs the difference was +0.076 with an interval of [-0.035, +0.184]. The other checks agree with this: with GLM-4.7-Flash the co-evolved system did not beat the fixed-Red baseline on recall, although three runs cannot settle it, a scripted evasive Red did not produce a clear benefit either, and with a larger rule cap the advantage over the fixed-Red baseline shrank to +0.145 with an interval that includes zero. The results do not support the claim that the feedback loop improves detection: the no-feedback ablation had clearly higher recall at both rule caps, and the feedback bought a lower false-positive rate at the standard cap (0.006 against 0.027) but not clearly at a cap of 20 (0.007 against 0.012). Which of the two is better depends on the cost of a missed attack against the cost of a false alert, which we do not model. We also cannot say that the gain over the fixed-Red baseline comes from the attacker responding to Blue, because the Red plans hardly depended on Blue. A possible explanation is that attacker variety, not adaptation, kept Blue exploring more patterns; the pooled-committee comparison did not separate co-evolved from fixed-Red runs once replicated (Section 6.6), so this explanation remains untested. A richer signal to Red did not change its plans either, so the 4-billion-parameter planner may be too insensitive to its memory for this design; we did not test a larger planner.

Implications for practice. A rule writer of this kind is useful as a proposal engine that is quiet by default, not as a replacement for broad hand-written coverage. Because about half of its rules never fire, a simple usage pattern is to prune silent rules after each round. Adding one run's rules to hand-written rules changed little, whereas pooling several independent runs raised recall a lot at a false-positive rate far below that of the hand-written rules (Section 6.6), so running several short loops and merging their rules looks more promising than relying on one run. This is a statement about LLM-written rules on this testbed, not a claim that they beat simpler methods: rule sets fitted without an LLM do better still here (Section 6-G), the hand-written set was untuned, and pooling uses about 590 rules against its ten.

9 Threats to Validity and Limitations

(1) Blue is detection-only: rules label events but do not change the simulated environment, so we measure rule quality, not defence quality. (2) Part of the telemetry is synthetic, written by us from successful Red actions; we report it separately because it is easy. (3) Two quantised models were used, the second for only three runs per condition, so model-size effects are not established. (4) We used ten runs of eight rounds per condition, so intervals are still wide, and the static baseline has one run. (5) The seen Red strategies were chosen by us, and the unseen strategy may share behaviour with them. (6) The rule language has no counts or time windows, which limits what any writer, human or model, can express. (7) Ground-truth labels come from our own labeller and were not independently audited. (8) Validity of a rule is not deployability: we have not tested rules on real telemetry or measured analyst workload. (9) Several design choices (the

retirement threshold, the rule cap, the prompt) were set during development, and conclusions about feedback may depend on them. (10) Ten runs per condition proved too few to settle the co-evolution comparison: ten new seeds reversed the ordering of the first ten, and only the two main conditions were replicated. (11) The testbed is easy to separate: attackers are 83.8% of native test events, benign network events all use ephemeral ports, and rule sets fitted on a few fields reach recall 0.94 to 1.00 at no false positives, so the results do not show that LLM-written rules help against harder, more overlapping traffic. (12) Native events come from only three Red actions, and part of the fields (process names, parent processes and similar) are filled in by a seeded rule that we wrote, so Blue may partly learn our data generator; with the enriched fields blanked, the co-evolved recall fell from 0.49 to 0.19 (the learned set from 1.00 to 0.93).

10 Conclusion and Future Work

We built a co-evolutionary setup in which LLM agents write Sigma-style detection rules for a simulated enterprise network while an LLM planner varies the attacker. In a pilot with a small local model, the rules were precise but incomplete: native precision of at least 0.97 and false-positive rates of at most 0.03, with native recall of 0.49 for the full system against 0.26 for a fixed attacker and 0.96 for ten hand-written rules. In the ten main runs co-evolution gave higher recall than a fixed attacker, but this did not replicate in ten further runs per condition (combined +0.076 [-0.035, +0.184]), in a three-run check with a larger model, with a scripted evasive Red, or with a larger rule budget, and performance feedback did not raise recall: an ablation without it reached 0.67 recall at a higher false-positive rate, and with a larger rule cap its recall was 0.72 with the false-positive gap largely closed. Pooling the rules of independent runs raised recall to 0.69 (three runs), 0.82 (five) and 0.96 (ten) at false-positive rates of 0.024, 0.045 and 0.080 in an exploratory analysis replicated on twenty runs per condition, although it did not show that co-evolved runs pool better than fixed-Red runs. A baseline check showed that rule sets fitted without an LLM reach recall 0.94 to 1.00 at no false positives on this testbed, whose native test events are 84% attacker events, so the testbed is easy to separate and the LLM rules are not shown to beat simpler methods. The numbers are small-scale and come from one simulator; we publish them with their limitations to guide the larger study that follows.

The next steps are to test a stronger rule writer, add counts and time windows to the language, wire a Blue response into the environment so that detection can be linked to outcome, and repeat the study with an attacker whose plans demonstrably respond to being caught, for example a larger planner model or a learned adversary that is rewarded for evading the current rules.

Appendix A. Experimental Configuration

Table 12 lists the settings shared by all runs.

Table 12: Settings used for the main experiments.

Parameter	Value
Simulator	CAGE Challenge 4 (CybORG), enterprise scenario
Episode length	500 steps; one warm-up episode before round 1
Rounds per training run	8
Training seeds (runs per condition)	0 to 9 (10)
Test seeds	100 to 109 (10), five Red strategies, 50 episodes per rule set
Blue agents	5, one LLM call per agent per round
Rules proposed per agent per round	up to 5
Active rules per agent	at most 12 (a cap of 20 in one variant)
Retirement rule	at least 3 fires and under 30% precision on attacker activity
Blue model	Qwen3-4B-Instruct-2507, 4-bit, temperature 0.4
Red planner model	same model, temperature 0.7, one call per round
Larger-model check	GLM-4.7-Flash, 3-bit, three runs per condition
Serving	llama.cpp, 4 parallel slots, one RTX 4050 (6 GB)
Bootstrap	95% percentile interval over runs; 4,000 resamples per condition, 10,000 for differences

Appendix B. Design Variants Tried

Before the final higher-cap run (Section 6.5) we tried three Blue-side variants on five development seeds (10 to 14) that are disjoint from the training seeds and the test seeds. The variants were the baseline prompt, a higher cap on active rules per agent (20 instead of 12), and a generic coverage hint added to the prompt (cover every event category in the zone's common patterns, and prefer one rule per distinct pattern). We fixed the selection rule beforehand: adopt a variant only if native training-episode recall rises by at least 0.05 and FPR stays within 0.01 of the baseline. Table 13 gives the screen. These are training-episode numbers over five seeds and serve only to choose a variant; they are not evidence.

Table 13: Development screen of Blue-side variants (five dev seeds, mean of rounds 7 and 8 per seed, native events).

Variant	Recall	FPR	Precision	Active rules
Baseline (cap 12)	0.516	0.016	0.994	about 60
Higher cap (20)	0.600	0.023	0.993	about 99
Coverage hint	0.528	0.065	0.966	about 60

Appendix C. Time to Detect

Time to detect is the number of steps between the first attacker event and the first alert on an attacker event in an episode. Table 14 gives the share of seen test episodes with any such alert and the median delay, for the same 40 episodes and rule sets as Table 1.

Table 14: Campaign-level detection on the seen test episodes (native events; mean over runs of the per-run median delay).

Rule set	Runs	Episodes with an alert (%)	Median steps to first alert
Static hand-written	1	100	0.0
Random rules	10	100	0.0
LLM, fixed Red	10	100	6.5
LLM, no feedback	10	100	0.2
LLM, co-evolved	10	100	2.0
LLM, per-action signal	10	100	3.5

Appendix D. Committee Sweep

Table 15 lists the committee sweep behind Fig. 9. Groups are disjoint sets of independent runs of one condition (ten runs give five groups of two, three groups of three, two groups of five and one group of ten); the entries are means over groups.

Table 15: Pooling the rules of independent runs (seen test episodes, native events; mean over disjoint groups).

Condition	Runs pooled	Groups	Rules	Prec.	Recall	FPR
LLM, fixed Red	1	10	60	0.974	0.258	0.011
LLM, fixed Red	2	5	119	0.992	0.437	0.022
LLM, fixed Red	3	3	179	0.993	0.534	0.020
LLM, fixed Red	5	2	298	0.985	0.701	0.055
LLM, fixed Red	10	1	595	0.986	0.953	0.071
LLM, no feedback	1	10	59	0.992	0.671	0.027
LLM, no feedback	2	5	118	0.989	0.850	0.049
LLM, no feedback	3	3	176	0.984	0.887	0.073
LLM, no feedback	5	2	294	0.981	0.943	0.093
LLM, no feedback	10	1	589	0.971	0.969	0.149
LLM, co-evolved	1	10	59	0.996	0.491	0.006
LLM, co-evolved	2	5	118	0.995	0.725	0.011
LLM, co-evolved	3	3	177	0.997	0.841	0.013
LLM, co-evolved	5	2	294	0.994	0.907	0.028
LLM, co-evolved	10	1	589	0.989	0.949	0.056
Evasive Red	1	10	59	0.987	0.344	0.006
Evasive Red	2	5	119	0.984	0.481	0.012
Evasive Red	3	3	179	0.994	0.684	0.019
Evasive Red	5	2	297	0.993	0.844	0.029
Evasive Red	10	1	594	0.992	0.961	0.039
Evasive Red, no fb.	1	10	59	0.992	0.526	0.018
Evasive Red, no fb.	2	5	119	0.991	0.737	0.033
Evasive Red, no fb.	3	3	178	0.988	0.746	0.045
Evasive Red, no fb.	5	2	297	0.982	0.781	0.073
Evasive Red, no fb.	10	1	594	0.973	0.802	0.115

Table 16 repeats the committee sweep for the two main conditions with twenty independent runs each (the ten main runs plus ten new runs on seeds 15 to 24, same settings). Intervals are 95% bootstrap intervals over groups and are shown only where there are at least four groups.

Table 16: Replication of the committee sweep with twenty runs per condition (seen test episodes, native events; mean [95% bootstrap CI over groups]).

Condition	Pooled	Groups	Rules	Recall	FPR	Prec.
LLM, fixed Red	1	20	60	0.293 [0.226, 0.361]	0.016 [0.009, 0.024]	0.974
LLM, fixed Red	2	10	119	0.476 [0.395, 0.551]	0.033 [0.019, 0.047]	0.987
LLM, fixed Red	3	6	178	0.608 [0.505, 0.743]	0.048 [0.020, 0.076]	0.985
LLM, fixed Red	5	4	298	0.762 [0.666, 0.880]	0.072 [0.050, 0.089]	0.982
LLM, fixed Red	10	2	595	0.959	0.105	0.979
LLM, co-evolved	1	20	59	0.368 [0.287, 0.454]	0.009 [0.005, 0.014]	0.994
LLM, co-evolved	2	10	118	0.590 [0.458, 0.721]	0.018 [0.008, 0.028]	0.992
LLM, co-evolved	3	6	177	0.687 [0.553, 0.822]	0.024 [0.013, 0.036]	0.992
LLM, co-evolved	5	4	296	0.820 [0.653, 0.923]	0.045 [0.027, 0.066]	0.989
LLM, co-evolved	10	2	592	0.958	0.080	0.984

Appendix E. Glossary

Table 17 explains the terms used throughout the paper in plain language.

Table 17: Terms used in this paper.

Term	Meaning
Red and Blue	The attacking side and the defending side. Here Red is an agent that plans attacks, and Blue is a team of five agents that write detection rules.
Purple teaming	Red and Blue working inside one feedback loop, so that each side learns from the other.
Detection rule	A small, readable program (a Sigma-style JSON object) that decides whether one event deserves an alert.
Event	One line of telemetry produced from the simulator, described by sixteen visible fields such as process name, port and privilege.
Native, enriched, synthetic	Events taken straight from the simulator; fields we filled in with a seeded rule because the simulator lacks them; and events we invented so that some attacker actions can be detected at all.
Hidden label	The simulator's record of whether Red caused an event. It is used for training feedback and for scoring, and is never shown to Blue or to a frozen rule.
Precision	The share of alerts that were raised on attacker events.
Recall	The share of attacker events that raised an alert.
FPR (false-positive rate)	The share of benign events that raised an alert.
Frozen rules	The rule set after training, scored on held-out seeds with no further feedback.
Co-evolution	Red and Blue adapting to each other over repeated rounds.
Committee	The union of the rules of several independent training runs.
Seen and unseen Red	Attacker strategies used while developing the system, and one strategy kept back for testing only.

Author Contributions

*These authors contributed equally to this work and share equal standing as authors. B. Anand built and evaluated the Blue-side system (telemetry bridge, rule language and validator, Blue agents, retirement logic), and ran the experiments reported here on his hardware. K. Anand built the Red agents, the ground-truth labeller, the metrics module and the continuous-integration checks that the experiments rely on. P. Tanwar built the experiment runner and its contract, the baseline conditions, and the literature review. All authors discussed the design, reviewed the results, and edited and approved the final text, which was drafted with AI assistance (see Use of AI Tools).

Reproducibility and Availability

Seeds, conditions, prompts, thresholds and test protocol are fixed in the experiment scripts; every number in this paper is read from the result files they write. The code, the pinned environment (Docker image and package pins), the experiment scripts and all result files, including the frozen rule sets and the cached test episodes, are in the public repository github.com/bhavitanand/co-evolutionary-purple-teaming-cage4.
Re-scoring the released rule sets regenerates every table exactly; retraining calls a language model and will not reproduce the same rules.

Use of AI Tools

We used Claude (Anthropic) as a coding and writing assistant. It helped to implement and debug parts of the experiment code and analysis scripts and to draft and edit text. The research questions, the experimental design and conditions, the choice of metrics, the stopping rules, and the interpretation of results were set and reviewed by the authors, who also ran every experiment on their own hardware and checked every reported number against the result files. The language models inside the system under study (Qwen3-4B-Instruct-2507, GLM-4.7-Flash) are components of the experiments, not tools used to write this paper.

Acknowledgment

The Red agent, labeller, metrics module and runner build on earlier work by the three authors in the project repository.

References

[1] E. Hemberg, S. Moskal, U.-M. O'Reilly, E. Liu, and L. Fuller, "Evolutionary and coevolutionary multi-agent design choices and dynamics," in *Proc. Genetic and Evolutionary Computation Conf. Companion (GECCO '25 Companion)*, 2025, pp. 559-562, doi: 10.1145/3712255.3726712.

[2] S. R. Castro, R. Campbell, N. Lau, O. Villalobos, J. Duan, and A. A. Cardenas, "Large language models are autonomous cyber defenders," in *Proc. IEEE CAI Adaptive CyberDefense Workshop*, 2025, arXiv:2505.04843.

[3] A. J. Shaikh, N. D. Bastian, and A. Shah, "A red teaming framework for evaluating robustness of AI-enabled security orchestration, automation, and response systems," arXiv:2605.17075, 2026.

[4] J. Zhou, K. Li, J. Li, J. Kang, M. Hu, X. Wu, and H. Meng, "Purple-teaming LLMs with adversarial defender training," arXiv:2407.01850, 2024.

[5] H. Wang, M. Xu, Y. Guo, W. Han, H. W. Lim, and J. S. Dong, "RulePilot: An LLM-powered agent for security rule generation," arXiv:2511.12224, 2025.

[6] S. Ghaffarzadegan, B. Nour, M. Pourzandi, M. Debbabi, and C. Assi, "From threat intelligence to detection: Knowledge-driven enrichment and template-based rule grounding for automated Sigma rule generation," arXiv:2608.19011, 2026.

[7] Y. Schwartz, L. Benshimol, D. Mimran, Y. Elovici, and A. Shabtai, "LLMCloudHunter: Harnessing LLMs for automated extraction of detection rules from cloud-based CTI," arXiv:2407.05194, 2024.

[8] M. Xu et al., "ThreatPilot: Attack-driven threat intelligence extraction," arXiv:2412.10872, 2024.

[9] CAGE Challenge 4, "Cyber Autonomy Gym for Experimentation, Challenge 4," 2024. [Online]. Available: https://github.com/cage-challenge/cage-challenge-4

[10] A. Chakraborty, S. Ho, A. Cook, and M. Melendez, "CTI-REALM: Benchmark to evaluate agent performance on security detection rule generation capabilities," arXiv:2603.13517, 2026.

[11] H. Du, "GenAI-powered autonomous cyber offense-defense: An explainable LLM red-vs-blue simulation and self-defense framework," *J. Cyber Security*, 2026, doi: 10.32604/jcs.2026.075976.

[12] L. H. Park et al., "Toward an autonomous purple teaming framework for security and safety in large language models," in *Proc. IEEE Pacific Rim Int. Symp. Dependable Computing (PRDC)*, 2025.

[13] C. Guo, Y. Nie, C. Xie, Z. Lin, W. Guo, and B. Li, "BlueCodeAgent: A blue teaming agent enabled by automated red teaming for CodeGen AI," arXiv:2510.18131, 2025.

[14] P. He et al., "Co-RedTeam: Orchestrated security discovery and exploitation with LLM agents," arXiv:2602.02164, 2026.

[15] Y. T. Shen, K. Toyoda, and A. Leung, "ACEA: An adversarial co-evolution arena for head-to-head red-team and blue-team LLM testing," arXiv:2609.08256, 2026.

[16] SigmaHQ, "Sigma specification: A generic and open signature format for SIEM systems," 2024. [Online]. Available: https://github.com/SigmaHQ/sigma-specification

[17] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," MITRE Corporation, Tech. Rep., 2018.

[18] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[19] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1-26, 1979.

[20] W. D. Hillis, "Co-evolving parasites improve simulated evolution as an optimization procedure," *Physica D*, vol. 42, no. 1-3, pp. 228-234, 1990.

[21] M. A. Potter and K. A. De Jong, "A cooperative coevolutionary approach to function optimization," in *Proc. Parallel Problem Solving from Nature (PPSN III)*, 1994, pp. 249-257.

[22] G. Gerganov et al., "llama.cpp: LLM inference in C/C++," 2023-2026. [Online]. Available: https://github.com/ggml-org/llama.cpp

[23] M. Standen, M. Lucas, D. Bowman, T. J. Richer, J. Kim, and D. Marriott, "CybORG: An autonomous cyber operations research gym," arXiv:2108.09118, 2021.